

Window Functions in PostgreSQL: Introduction

Table of Contents

- 1. [Learning Objectives](#)
 - 2. [Theory: What are Window Functions?](#)
 - 3. [OVER Clause](#)
 - 4. [PARTITION BY](#)
 - 5. [ORDER BY in Window Context](#)
 - 6. [Frame Clauses: ROWS vs RANGE vs GROUPS](#)
 - 7. [UNBOUNDED PRECEDING and UNBOUNDED FOLLOWING](#)
 - 8. [Named Windows \(WINDOW clause\)](#)
 - 9. [Aggregate Functions as Window Functions](#)
 - 10. [ASCII Visual Diagrams](#)
 - 11. [Common Mistakes](#)
 - 12. [Best Practices](#)
 - 13. [Performance Considerations](#)
 - 14. [Interview Questions & Answers](#)
 - 15. [Hands-on Exercises](#)
 - 16. [Advanced Notes](#)
 - 17. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Understand what window functions are and how they differ from aggregates
 - Write queries using OVER, PARTITION BY, and ORDER BY
 - Define precise frame clauses with ROWS, RANGE, and GROUPS
 - Use aggregate functions as window functions for running totals
 - Explain window function execution order in interview settings
-

Theory: What are Window Functions?

A window function performs a calculation across a set of related rows (a "window") without collapsing them into a single output row. Unlike GROUP BY aggregations, each row retains its individual identity while having access to aggregated values from related rows.

Window Function vs GROUP BY

GROUP BY (aggregation):	Window Function:
Input: 5 rows	Input: 5 rows
Output: 1 row (collapsed)	Output: 5 rows (each row preserved)
SELECT dept, AVG(salary) FROM employees GROUP BY dept;	SELECT dept, salary, AVG(salary) OVER (PARTITION BY dept) FROM employees;

Logical Processing Order

Window functions are evaluated AFTER WHERE, GROUP BY, and HAVING — but BEFORE ORDER BY and LIMIT:

```
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT (window functions here) → ORDER BY → LIMIT
```

This means you cannot filter on a window function result in WHERE — use a subquery or CTE.

OVER Clause

The OVER clause defines the window. An empty OVER() means the window is the entire result set.

Syntax

```
function_name(expression) OVER (  
    [PARTITION BY partition_expression]  
    [ORDER BY order_expression [ASC|DESC] [NULLS FIRST|LAST]]  
    [frame_clause]  
)
```

Setup Tables

```
CREATE TABLE emp_salary (  
    emp_id        SERIAL PRIMARY KEY,  
    emp_name      VARCHAR(50),  
    department    VARCHAR(30),  
    hire_date     DATE,  
    salary        NUMERIC(10,2),  
    manager_id    INT  
);  
  
INSERT INTO emp_salary (emp_name, department, hire_date, salary, manager_id) VALUES  
    ('Alice',      'Engineering', '2020-03-15', 95000, NULL),  
    ('Bob',        'Engineering', '2021-06-01', 87000, 1),  
    ('Carol',      'Engineering', '2019-11-20', 105000, 1),  
    ('David',      'Marketing',   '2020-08-10', 72000, NULL),  
    ('Eve',        'Marketing',   '2022-01-15', 68000, 4),  
    ('Frank',      'Marketing',   '2021-09-30', 75000, 4),  
    ('Grace',      'Sales',       '2018-05-22', 82000, NULL),  
    ('Henry',      'Sales',       '2022-03-01', 65000, 7),  
    ('Iris',       'Sales',       '2020-07-14', 78000, 7),  
    ('Jack',       'Sales',       '2023-01-10', 61000, 7);  
  
CREATE TABLE monthly_revenue (  
    month_date    DATE,  
    department    VARCHAR(30),  
    revenue       NUMERIC(12,2)  
);
```

```

INSERT INTO monthly_revenue VALUES
  ('2024-01-01', 'Engineering', 45000),
  ('2024-02-01', 'Engineering', 52000),
  ('2024-03-01', 'Engineering', 48000),
  ('2024-04-01', 'Engineering', 61000),
  ('2024-01-01', 'Marketing', 28000),
  ('2024-02-01', 'Marketing', 31000),
  ('2024-03-01', 'Marketing', 35000),
  ('2024-04-01', 'Marketing', 29000),
  ('2024-01-01', 'Sales', 38000),
  ('2024-02-01', 'Sales', 42000),
  ('2024-03-01', 'Sales', 45000),
  ('2024-04-01', 'Sales', 51000);

```

Examples

```

-- Example 1: OVER() – global average alongside each row
SELECT
  emp_name,
  department,
  salary,
  ROUND(AVG(salary) OVER (), 2) AS company_avg,
  salary - ROUND(AVG(salary) OVER (), 2) AS diff_from_avg
FROM emp_salary
ORDER BY department, salary DESC;

-- Example 2: COUNT with OVER() – total employee count on every row
SELECT
  emp_name,
  department,
  salary,
  COUNT(*) OVER () AS total_employees
FROM emp_salary;

```

PARTITION BY

PARTITION BY divides rows into groups. The window function is applied independently within each partition.

ASCII Diagram

Without PARTITION BY:

All rows = one window

AVG(salary) OVER ()
= global average

With PARTITION BY department:

Engineering rows = one window
Marketing rows = one window
Sales rows = one window

AVG(salary) OVER (PARTITION BY department)
= dept average per row

Examples

```
-- Example 3: Department average alongside each employee
SELECT
    emp_name,
    department,
    salary,
    ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS dept_avg,
    salary - ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS vs_dept_avg
FROM emp_salary
ORDER BY department, salary DESC;

-- Example 4: Department headcount and salary range
SELECT
    emp_name,
    department,
    salary,
    COUNT(*) OVER (PARTITION BY department) AS dept_headcount,
    MIN(salary) OVER (PARTITION BY department) AS dept_min,
    MAX(salary) OVER (PARTITION BY department) AS dept_max,
    SUM(salary) OVER (PARTITION BY department) AS dept_salary_total
FROM emp_salary
ORDER BY department, salary DESC;

-- Example 5: Partition by multiple columns
SELECT
    emp_name,
    department,
    hire_date,
    salary,
    AVG(salary) OVER (
        PARTITION BY department,
            EXTRACT(YEAR FROM hire_date)
    ) AS cohort_avg_salary
FROM emp_salary
ORDER BY department, hire_date;
```

ORDER BY in Window Context

When ORDER BY is added to OVER, the window function sees rows in order. For aggregate functions (SUM, AVG, etc.), this enables running totals. The default frame becomes RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

Examples

```
-- Example 6: Running total – cumulative revenue per department
SELECT
    department,
    month_date,
```

```

    revenue,
    SUM(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
    ) AS cumulative_revenue
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 7: Running average – smoothing
SELECT
    department,
    month_date,
    revenue,
    ROUND(AVG(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
    ), 2) AS running_avg
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 8: Running COUNT – how many months processed so far
SELECT
    department,
    month_date,
    revenue,
    COUNT(*) OVER (
        PARTITION BY department
        ORDER BY month_date
    ) AS months_processed
FROM monthly_revenue
ORDER BY department, month_date;

```

Frame Clauses

The frame clause defines the exact set of rows within the partition that the window function considers. It only applies when ORDER BY is specified.

Frame Types

```

ROWS    – frame defined by physical row position offsets
RANGE   – frame defined by logical value range relative to current row value
GROUPS  – frame defined by peer groups (rows with equal ORDER BY values)

```

Frame Boundaries

```

UNBOUNDED PRECEDING  – from the very first row of the partition
N PRECEDING          – N rows/units before the current row
CURRENT ROW          – the current row

```

N FOLLOWING – N rows/units after the current row
UNBOUNDED FOLLOWING – to the very last row of the partition

ASCII Diagram — Frame Visualization

Partition rows: R1, R2, R3(current), R4, R5

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW:
[R1, R2, R3] ← includes up to current

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING:
[R2, R3, R4] ← sliding window of 3 rows

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING:
[R3, R4, R5] ← current to end

ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING:
[R1, R2, R3, R4, R5] ← entire partition

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW (default with ORDER BY):
All rows where ORDER BY value ≤ current row's value
(includes peers – rows with same value as current)

Examples

```
-- Example 9: ROWS vs RANGE – the difference matters with ties
SELECT
    month_date,
    revenue,
    SUM(revenue) OVER (ORDER BY revenue
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_sum,
    SUM(revenue) OVER (ORDER BY revenue
        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS range_sum
FROM monthly_revenue
WHERE department = 'Engineering';
-- If two months have equal revenue, RANGE includes all ties together
-- ROWS includes only up to the physical current row

-- Example 10: Moving average (3-month rolling window)
SELECT
    department,
    month_date,
    revenue,
    ROUND(AVG(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS moving_avg_3m
FROM monthly_revenue
ORDER BY department, month_date;
```

-- Example 11: Centered moving window (previous, current, next)

```
SELECT
    department,
    month_date,
    revenue,
    ROUND(AVG(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ), 2) AS centered_avg
FROM monthly_revenue
ORDER BY department, month_date;
```

-- Example 12: Cumulative sum (default behavior with ORDER BY)

```
SELECT
    department,
    month_date,
    revenue,
    SUM(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_sum
FROM monthly_revenue
ORDER BY department, month_date;
```

-- Example 13: Reverse cumulative (current to end)

```
SELECT
    department,
    month_date,
    revenue,
    SUM(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
    ) AS remaining_revenue
FROM monthly_revenue
ORDER BY department, month_date;
```

-- Example 14: Max of last 3 rows including current

```
SELECT
    department,
    month_date,
    revenue,
    MAX(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS max_last_3
```

```
FROM monthly_revenue
ORDER BY department, month_date;
```

UNBOUNDED PRECEDING and UNBOUNDED FOLLOWING

```
-- Example 15: Full partition sum (denominator for percentage)
SELECT
    department,
    month_date,
    revenue,
    SUM(revenue) OVER (PARTITION BY department) AS dept_total,
    ROUND(100.0 * revenue /
          SUM(revenue) OVER (PARTITION BY department), 2) AS pct_of_dept
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 16: Global running percentage
SELECT
    month_date,
    department,
    revenue,
    SUM(revenue) OVER (ORDER BY month_date, department
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS global_running_total,
    SUM(revenue) OVER (
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS grand_total
FROM monthly_revenue
ORDER BY month_date, department;
```

Named Windows (WINDOW clause)

When you use the same window definition multiple times, define it once with the WINDOW clause.

```
-- Example 17: Named window – reuse the same partition definition
SELECT
    emp_name,
    department,
    salary,
    AVG(salary) OVER dept_window AS dept_avg,
    MAX(salary) OVER dept_window AS dept_max,
    MIN(salary) OVER dept_window AS dept_min,
    COUNT(*) OVER dept_window AS dept_count,
    SUM(salary) OVER dept_window AS dept_total_payroll
FROM emp_salary
WINDOW dept_window AS (PARTITION BY department)
ORDER BY department, salary DESC;

-- Example 18: Named window with ORDER BY
```



```

SELECT
    department,
    month_date,
    revenue,
    SUM(revenue) OVER w AS running_total,
    AVG(revenue) OVER w AS running_avg,
    COUNT(*) OVER w AS months_so_far
FROM monthly_revenue
WINDOW w AS (PARTITION BY department ORDER BY month_date
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
ORDER BY department, month_date;

```

Aggregate Functions as Window Functions

Any aggregate function can be used as a window function by adding an OVER clause.

```

-- Example 19: All aggregates as window functions
SELECT
    emp_name,
    department,
    salary,
    COUNT(*) OVER (PARTITION BY department) AS dept_count,
    SUM(salary) OVER (PARTITION BY department) AS dept_payroll,
    AVG(salary) OVER (PARTITION BY department) AS dept_avg,
    MIN(salary) OVER (PARTITION BY department) AS dept_min,
    MAX(salary) OVER (PARTITION BY department) AS dept_max,
    -- Percentage of dept payroll
    ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY department), 2) AS
pct_payroll
FROM emp_salary
ORDER BY department, salary DESC;

```

ASCII Visual Diagrams

Window Function Architecture

WINDOW FUNCTION EXECUTION

Input rows (after WHERE/GROUP BY/HAVING):

emp	dept	salary	
Alice	Engineering	95000	
Bob	Engineering	87000	
Carol	Engineering	105000	
David	Marketing	72000	
Eve	Marketing	68000	
Frank	Marketing	75000	

...	Sales	...	→ Sales window

Output: Same number of rows, each with window result appended

Frame Clause Cheat Sheet

Frame boundary keywords:

UNBOUNDED PRECEDING	: start of partition
N PRECEDING	: N rows/values before current
CURRENT ROW	: this row (inclusive)
N FOLLOWING	: N rows/values after current
UNBOUNDED FOLLOWING	: end of partition

Defaults:

OVER ()	→ entire partition
OVER (ORDER BY col)	→ RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
OVER (PARTITION BY p ORDER BY col)	→ RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Common Mistakes

Mistake 1: Filtering on window function result in WHERE

```
-- WRONG: window functions not evaluated yet at WHERE time
SELECT emp_name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk
FROM emp_salary
WHERE RANK() OVER (ORDER BY salary DESC) <= 3; -- ERROR

-- CORRECT: wrap in subquery or CTE
SELECT * FROM (
    SELECT emp_name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM emp_salary
) AS ranked
WHERE rnk <= 3;
```

Mistake 2: Confusing ROWS and RANGE frame semantics with ties

```
-- With duplicate values in ORDER BY:
-- RANGE BETWEEN ... CURRENT ROW includes ALL tied rows
-- ROWS BETWEEN ... CURRENT ROW includes only up to the physical row
-- For running totals, usually use ROWS for deterministic behavior
```

Mistake 3: Missing PARTITION BY — computing global instead of group value

```
-- WRONG: computes global average, not dept average
AVG(salary) OVER (ORDER BY salary) -- no PARTITION BY

-- CORRECT:
AVG(salary) OVER (PARTITION BY department ORDER BY salary)
```

Mistake 4: Using window function in GROUP BY or WHERE

```
-- Window functions cannot be used in GROUP BY, WHERE, or HAVING
-- They can only be used in SELECT and ORDER BY clauses
```

Best Practices

1. Use WINDOW clause when reusing the same window definition in multiple functions.
2. Prefer ROWS over RANGE for running totals when you want deterministic behavior.
3. Use explicit frame clauses — do not rely on the default RANGE semantics unexpectedly.
4. Wrap window function results in a subquery/CTE before filtering.
5. Use PARTITION BY instead of correlated subqueries for group-level calculations.
6. Avoid overly complex frames — keep window specs simple and comment them.

Performance Considerations

```
-- Window functions require sorting the input (when ORDER BY is specified)
-- Each unique OVER() definition requires a separate sort operation
-- Consolidate windows when possible using WINDOW clause

-- Index on PARTITION BY and ORDER BY columns can help:
CREATE INDEX idx_emp_dept_salary ON emp_salary(department, salary DESC);

-- Check the plan for Sort and WindowAgg nodes:
EXPLAIN (ANALYZE, FORMAT TEXT)
SELECT emp_name,
       AVG(salary) OVER (PARTITION BY department ORDER BY salary)
FROM emp_salary;

-- PostgreSQL 13+: Incremental sorting for chained window functions
-- Can reuse partial sorts when ORDER BY prefixes match
```

Interview Questions & Answers

Q1: What is the difference between a window function and an aggregate function?

A: An aggregate function (with GROUP BY) collapses multiple rows into one output row per group. A window function performs calculations across related rows without collapsing them — each input row produces one output row, with the aggregated value appended.

Q2: What does PARTITION BY do in a window function?

A: PARTITION BY divides the result set into independent groups (partitions), and the window function is applied separately within each partition. Without PARTITION BY, the entire result set is one window.

Q3: What is a frame clause and why does it matter?

A: The frame clause defines which rows within the partition are visible to the window function. It controls whether you get a running total (UNBOUNDED PRECEDING to CURRENT ROW), a sliding window (N PRECEDING to N FOLLOWING), or a total for the whole partition. Without understanding frame clauses, running totals can behave unexpectedly with duplicate values.

Q4: What is the difference between ROWS and RANGE frame types?

A: ROWS defines the frame by physical row position offsets. RANGE defines the frame by logical value range — all rows with ORDER BY values within the specified range of the current row's value. With ties (equal ORDER BY values), RANGE includes all tied rows in the frame together, while ROWS counts each row individually.

Q5: Why can you not use a window function result in a WHERE clause?

A: Window functions are evaluated after WHERE (step 6 of logical processing order). The WHERE clause is evaluated at step 3. To filter on a window function result, wrap the query in a subquery or CTE.

Q6: What does an empty OVER() clause mean?

A: An empty OVER() means no partitioning and no ordering — the entire result set is one window. The function operates over all rows. No frame clause applies without ORDER BY.

Q7: How does RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW differ from ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW?

A: With RANGE, "CURRENT ROW" includes all peer rows (rows with the same ORDER BY value). With ROWS, "CURRENT ROW" includes only the single physical current row. They differ when there are duplicate ORDER BY values.

Q8: What is the WINDOW clause for?

A: The WINDOW clause lets you define a named window specification that can be referenced by multiple window functions in the SELECT, avoiding repetition and ensuring consistency.

Hands-on Exercises

Exercise 1: Department Statistics

For each employee, show their salary alongside the department's average, maximum, and their salary's percentage of the department total.

```
-- Solution:
SELECT
    emp_name,
    department,
    salary,
    ROUND(AVG(salary) OVER (PARTITION BY department), 2) AS dept_avg,
    MAX(salary) OVER (PARTITION BY department) AS dept_max,
    ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY department), 2) AS
    pct_of_dept
```

```
FROM emp_salary
ORDER BY department, salary DESC;
```

Exercise 2: 3-Month Rolling Average

For the monthly_revenue table, calculate a 3-month rolling average revenue per department.

```
-- Solution:
SELECT
    department,
    month_date,
    revenue,
    ROUND(AVG(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS rolling_avg_3m,
    COUNT(*) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS window_size
FROM monthly_revenue
ORDER BY department, month_date;
```

Exercise 3: Named Window

Rewrite the department statistics query using a named WINDOW clause.

```
-- Solution:
SELECT
    emp_name,
    department,
    salary,
    COUNT(*) OVER dept AS dept_count,
    SUM(salary) OVER dept AS dept_payroll,
    AVG(salary) OVER dept AS dept_avg,
    MIN(salary) OVER dept AS dept_min,
    MAX(salary) OVER dept AS dept_max
FROM emp_salary
WINDOW dept AS (PARTITION BY department)
ORDER BY department, salary DESC;
```

Exercise 4: Cumulative Share

Show how each month's revenue contributes cumulatively to the yearly total for each department.

```
-- Solution:
SELECT
    department,
```

```

month_date,
revenue,
SUM(revenue) OVER (
    PARTITION BY department
    ORDER BY month_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS cumulative_revenue,
SUM(revenue) OVER (PARTITION BY department) AS annual_total,
ROUND(100.0 *
    SUM(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) / SUM(revenue) OVER (PARTITION BY department), 2
) AS cumulative_pct
FROM monthly_revenue
ORDER BY department, month_date;

```

Advanced Notes

Window Functions on Aggregated Results

Window functions can operate on aggregated results when used in a query with GROUP BY:

```

-- First GROUP BY, then window over the groups
SELECT
    department,
    DATE_TRUNC('month', hire_date) AS hire_month,
    COUNT(*) AS hires,
    SUM(COUNT(*)) OVER (
        PARTITION BY department
        ORDER BY DATE_TRUNC('month', hire_date)
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_hires
FROM emp_salary
GROUP BY department, DATE_TRUNC('month', hire_date)
ORDER BY department, hire_month;

```

EXCLUDE in Frame Clause (PostgreSQL 12+)

```

-- EXCLUDE CURRENT ROW – exclude the current row from the frame
SELECT emp_name, salary,
    AVG(salary) OVER (
        PARTITION BY department
        ORDER BY salary
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        EXCLUDE CURRENT ROW -- average of all others in partition
    ) AS avg_excluding_self
FROM emp_salary;

```

Cross-references

- **02_ranking_functions.md** — ROW_NUMBER, RANK, DENSE_RANK, NTILE
- **03_analytical_functions.md** — LAG, LEAD, FIRST_VALUE, LAST_VALUE
- **06_advanced_aggregation.md** — Statistical window functions
- **08_interview_window_tasks.md** — 20+ real company window function problems
- **02_aggregations.md** (03_Intermediate_SQL) — GROUP BY as the non-window alternative